

자유스터디 활동보고서(3차)

※ 빨간색 글씨를 참고한 뒤 삭제 후 작성 바랍니다.

팀명	KHUDA	학습 주제	딥러닝과 자연어처리	
교과목명/ 교수자명	딥러닝과 자연어처리			
학습 일시	2025년 5월 4일 오후 3시 ~ 2025년 5월 4일 오후 5시		학습 시간	총 2시간
진행 방식 (소통 도구)	Google meet			
참석자	성명		인원	
	김재욱, 김민서, 오종현, 한지훈, 조민우		총 5명	
회차별 학습 주제	Batch normalization, Residual connection			
세부 학습 내용 (3가지 이상)	학습 내용			
	1. 초기 Loss 및 saturation 현상			
	학습 시작 시 초기 loss 값을 확인하여 모델이 정상적으로 학습을 시작하는지 점검한다. 초기 loss가 지나치게 크거나 작을 경우, 네트워크가 포화(saturation) 상태에 있을 수 있음을 의심할 수 있다. 특히, 활성화 함수의 출력이 극단적인 값(예: ReLU의 경우 0, sigmoid의 경우 0 또는 1)에 치우쳐 있을 때 이러한 현상이 발생한다. Batch Normalization을 도입함으로써 입력 분포를 정규화하여 각 층의 활성화가 적절한 범위 내에 분포하도록 하여, 포화 현상을 효과적으로 완화할 수 있다.			
	2. Gradient 분포 및 Vanishing Gradient			
	학습 과정에서 각 레이어의 gradient 분포를 모니터링하여 vanishing gradient 문제를 점검한다. 깊은 신경망에서는 역전파 과정에서 gradient가 점점 작아져 앞단 레이어까지 제대로 전달되지 않는 현상이 발생할 수 있다. Residual Connection을 적용함으로써, gradient가 shortcut을 통해 직접 전달되어 기울기 소실 문제가 크게 완화됨을 확인했다. 실제로, gradient 분포가 0에 수렴하지 않고 일정 수준 이상으로 유지되었다.			
세부 학습 내용 (3가지 이상)	3. Loss 그래프 및 Learning Rate			
	Learning rate 적절한지 확인하기 위해, 학습 중 loss 곡선과 gradient:weight 비율을 함께 확인했다. Loss가 급격히 진동하거나 발산하는 경우, 학습률이 너무 높을 수 있다. 반대로 loss가 거의 감소하지 않는 경우, 학습률이 너무 낮을 수 있다. Gradient:weight 비율이 너무 크거나 작지 않은지 확인함으로써, 파라미터가 적절한 속도로 업데이트되고 있음을 확인했고, 이를 통해 적절한 learning rate를 설정하였고, 안정적인 학습 곡선을 얻을 수 있음을 확인했다.			
	4. Train-Validation Loss 모니터링을 통한 Overfitting			
	모델의 일반화 성능을 확인하기 위해, 학습 데이터와 검증 데이터의 loss를 동시에 확인했다. 학습 데이터의 loss는 지속적으로 감소하지만 검증 데이터의 loss가 일정 시점 이후 증가하는 경우, 과적합(overfitting)이 발생하고 있음을 알 수 있다. Batch Normalization과 Residual Connection의 도입으로 모델의 일반화 성능이 향상되어, train-loss와 val-loss 간의 차이가 줄어드는 경향을 확인했다. 또한, 필요에 따라 조기 종료(early stopping) 등의 추가적인 regularization 기법도 적용할			

수 있다.

5. Word Embedding 점검

텍스트 데이터를 다루는 경우, embedding layer에서 학습된 단어 임베딩의 품질을 점검했다. 임베딩 벡터 간 코사인 유사도를 측정하거나, 임베딩 벡터를 시각화하여 클러스터링이 잘 이루어지는지 확인해야하며. 이를 통해 embedding layer가 의미 있는 표현을 학습하고 있는지 검증하고, 필요시 임베딩 차원이나 학습률 등의 하이퍼파라미터를 조정해야한다.

회차별
팀원 역할
인당 1줄

김재욱 : 3회차 스터디 이론 발제

조민우 : 3회차 스터디 시간 조율 및 공지, 활동보고서 작성

한지훈 : 3회차 스터디 과제 공지 및 안내

김민서 : 3회차 스터디 실습 코드 작성 및 풀이

오종현 : 지난 회차 스터디의 과제 풀이

학습 활동
증빙자료

학습
자료

```

@dataclass
class MLPConfig:
    root_dir: str = os.getcwd() + "/../.."
    dataset_path: str = "data/names.txt"
    device: torch.device = torch.device("cpu") # Automatic device configuration

    # Tokenizer
    vocab_size: int = 0 # Set later

    # Model
    context_size: int = 3
    d_embed: int = 16
    d_hidden: int = 256

    # Training
    val_size: float = 0.1
    batch_size: int = 32
    max_steps: int = 1000
    lr: float = 0.01
    val_interval: int = 100

    seed: int = 101

config = MLPConfig()

set_seed(config.seed)

Random seed set to 101

config.device = configure_device()

Running on 1 NVIDIA GeForce RTX 4080 SUPER GPU(s)

# Train
def train():
    model = nn.Module()
    train_loader = DataLoader(
        train_loader=train_loader,
        max_steps=int,
        lr=float,
        val_interval=int,
        device=torch.device
    )

    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    steps = 1
    train_losses = []
    val_losses = []
    train_iter = itertools.cycle(train_loader) # Infinite dataloader

    for step in range(1, max_steps + 1):
        model.train()
        train_inputs, train_targets = next(train_iter)
        train_inputs, train_targets = train_inputs.to(device), train_targets.to(device)
        optimizer.zero_grad()
        logits = model(train_inputs)
        loss = F.cross_entropy(logits, train_targets)
        loss.backward()
        optimizer.step()
        steps.append(step)
        train_losses.append(loss.item())

    if step % val_interval == 0:
        model.eval()
        val_inputs, val_targets = next(train_iter)
        val_inputs, val_targets = val_inputs.to(device), val_targets.to(device)
        val_logits = model(val_inputs)
        batch_loss = F.cross_entropy(val_logits, val_targets)
        val_loss = batch_loss.item() * val_inputs.size(0)
        total_samples += val_inputs.size(0)
        val_losses.append(val_loss)
        print(f"Step {step}: Train Loss = {loss.item():.4f}, Val Loss = {val_loss:.4f}")

    if step == 1:
        print(f"Initial Train Loss = {loss.item():.4f}")

# Plot the loss
plt.figure()
plt.plot(steps, train_losses, label="Train")
val_steps = [step for step in steps if step % val_interval == 0]
plt.plot(val_steps, val_losses, label="Validation")
plt.xlabel("Steps")
plt.ylabel("Loss")
plt.legend()
plt.show()

```

```

for idx, (layer, bn) in enumerate(zip(model.layers[1:], model.batch_norms[1:]), start=2):
    residual = a
    a = layer(a)
    a = bn(a)
    a = F.tanh(a)
    if model.residual:
        a = a + residual
    activations["layer {idx}"] = a.detach().cpu().numpy()

return activations

# Sample a batch from the train_loader.
sample = next(iter(train_loader))[0]

# Get activations for both non-residual and residual models.
acts_no_res = get_activations(deep_mlp, sample)
acts_res = get_activations(deep_mlp_res, sample)

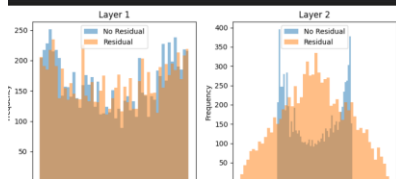
# Plot the activation Histograms.
import math
cols = 5
rows = math.ceil(len(acts_no_res) / cols)
fig, axes = plt.subplots(rows, cols, figsize=(20, 4 * rows))
axes = axes.flatten()

for idx, layer in enumerate(acts_no_res.keys()):
    axes[idx].hist(acts_no_res[layer].flatten(), bins=50, alpha=0.5, label='No Residual')
    axes[idx].hist(acts_res[layer].flatten(), bins=50, alpha=0.5, label='Residual')
    axes[idx].set_title(layer)
    axes[idx].set_xlabel("Activation Value")
    axes[idx].set_ylabel("Frequency")
    axes[idx].legend()

# Turn off any extra subplots.
for j in range(idx + 1, len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()

```



```

class NamesDataset(Dataset):
    def __init__(self, names, context_size):
        self.inputs, self.targets = [], []

        for name in names:
            context = [0] * context_size
            for char in name + " ":
                self.inputs.append(context)
                self.targets.append(char)
                context = context[1:] + [char]

    def __len__(self):
        return len(self.inputs)

    def __getitem__(self, idx):
        input_ids = torch.tensor(self.inputs[idx])
        target_id = torch.tensor(self.targets[idx])
        return input_ids, target_id

train_dataset = NamesDataset(train_names, config.context_size)
val_dataset = NamesDataset(val_names, config.context_size)
train_loader = DataLoader(train_dataset, batch_size=config.batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=config.batch_size, shuffle=False)

class MLP(nn.Module):
    def __init__(self, vocab_size, context_size, d_embed, d_hidden):
        super().__init__()
        self.fc = nn.Parameter(torch.randn(vocab_size, d_embed))
        self.M1 = nn.Parameter(torch.randn(context_size + d_embed, d_hidden))
        self.b1 = nn.Parameter(torch.randn(d_hidden))
        self.M2 = nn.Parameter(torch.randn(d_hidden, vocab_size))
        self.b2 = nn.Parameter(torch.randn(vocab_size))

    def forward(self, x): # x = [batch_size, context_size]
        # Embedding
        x_embed = self.fc[x] # [batch_size, context_size, d_embed]
        x = x_embed.view(-1, context_size + d_embed)

        # Hidden layer
        h = F.tanh(x @ self.M1 + self.b1) # [batch_size, d_hidden]

        # Output layer
        logits = h @ self.M2 + self.b2 # [batch_size, vocab_size]
        return logits

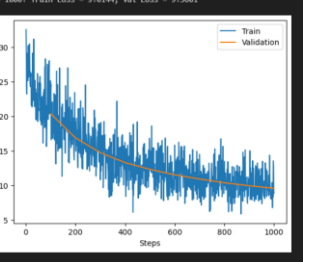
```

```

train(
    model=mlp,
    train_loader=train_loader,
    val_loader=val_loader,
    max_steps=config.max_steps,
    lr=config.lr,
    val_interval=config.val_interval,
    device=config.device
)

Initial Train Loss = 32.5348
Step 100: Train Loss = 24.1774, Val Loss = 20.3688
Step 200: Train Loss = 19.1782, Val Loss = 16.8962
Step 300: Train Loss = 18.7864, Val Loss = 14.7725
Step 400: Train Loss = 14.2168, Val Loss = 13.2787
Step 500: Train Loss = 12.1662, Val Loss = 12.2998
Step 600: Train Loss = 15.9862, Val Loss = 11.5175
Step 700: Train Loss = 14.4778, Val Loss = 10.9688
Step 800: Train Loss = 11.4083, Val Loss = 10.3655
Step 900: Train Loss = 10.6566, Val Loss = 9.9281
Step 1000: Train Loss = 9.8144, Val Loss = 9.5681

```



	활동 사진 (1장)	
--	------------------	--

학습 자료 첨부